Homework 3 - Nonparametric Methods, Ensemble Methods

Instructions

Due November 6 at 11pm

Instructions

- Use the space inside of

```
::: {.solbox data-latex=""}


:::
```

to answer the following questions.

- Do not move this file or copy it and work elsewhere. Work in the same directory.
- Use a new branch named whatever you want. Create it now! Make a small change, say by adding your name. Commit and push now!
- Try to Knit your file now. Are there errors? Fix them now, not at 11pm on the due date.
- There MUST be some text between `::: {.solbox}` and the next `:::` or this will fail to Knit.
- If your code or figures run off the edge of the `.pdf`, you'll lose 2 points automatically.
- Be sure to add your name in the `author` field at the top.

Generative AI Self Report

If you use any generative AI tool on homeworks or labs, indicate here how you used the tools and how they contributed to your learning/ability to understand the assignment. Include every prompt that you used.

This self report will not affect your grade in any way. Please be as honest as possible.

- your response here *

1 Nonparametric regression (3 points)

```r
data("arcuate")

# Split the arcuate data into training and test splits
set.seed(12345)
arcuate.train <- arcuate %>% slice_sample(n = 10, replace = FALSE)
arcuate.test <- arcuate %>% anti_join(arcuate.train, by = "position")
```

1.1 (2 points)

You will now implement kernel ridge regression on the arcuate data. The kernel you will use will be the radial basis function (RBF) kernel, which is implemented below:

```r
# @param x1: A vector representing the first input x1
# @param x2: A vector representing the second input x2
# @param bw: A float representing the bandwidth for the RBF kernel
# Computes the kernel value k(x1, x2) = exp(-0.5 * (x1 - x2)^2 / bw^2)
rbf_kernel <- function(x1, x2, bw) {
  exp(-0.5 * (x1 - x2)^2 / bw^2)
}
```

a. (1 point) Complete the function below that computes predictions on test data using kernel ridge regression with the RBF kernel.

b. (1 point) Use the function you just wrote to compute predictions on the test data using a bandwidth of 30 and a regularization parameter of 0.01. Plot the predictions alongside the test and training data.

Important hints - Given matrix A and vector y, computing $A^{-1}y$ can be numerically unstable if you do it wrong. The best approach in R is to use the function `solve(A, y)`. - To avoid using a slow `for` loop to construct kernel matrices, consider using the `outer` function.

a.

```r
# Computes predictions on test data using kernel ridge regression with the
#   RBF kernel.
# @param x.train: A vector containing the training input
#   (i.e. `arcuate.train$position`)
# @param y.train: A vector containing the training responses
#   (i.e. `arcuate.train$fa`)
# @param x.test: A vector containing the test input
#   (i.e. `arcuate.test$position`)
# @param bw: A float representing the bandwidth for the RBF kernel
# @param lambda: A float representing the regularization parameter
# @return A vector containing the predictions for the test data
krr <- function(x.train, y.train, x.test, bw = 0.5, lambda = 0.1) {
  # your code here
}
```

b.

1.2 (1 point)

Try decreasing the bandwidth parameter and plotting the predictions. Answer the following questions (no code required):

a. (0.5 points) Describe what happens to predictions that are far away from the training data as you decrease the bandwidth parameter. Is this behaviour desirable?
b. (0.5 points) What is a simple transformation you can apply to your responses to make the predictions more reasonable?

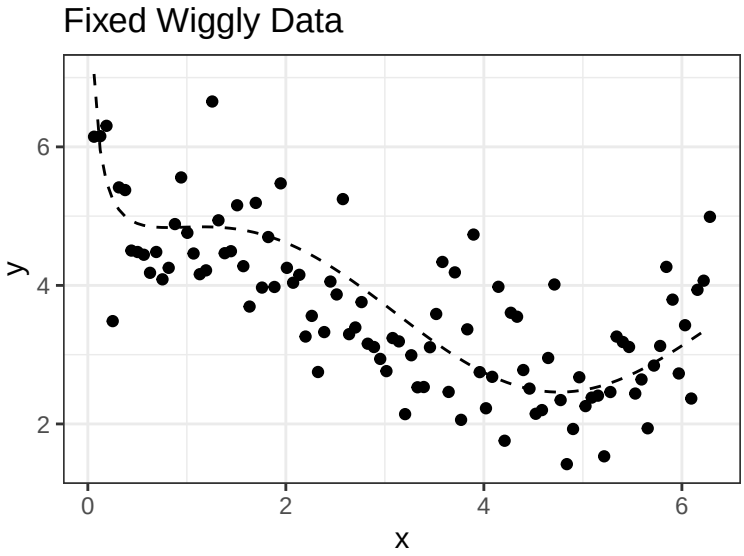
a.
b.
c.

2 Bootstrapping (4 points)

We will revisit the `wiggly_function` data from Homework 2 Question 2. This time, rather than simulating training samples, we will instead use a fixed training set of $n = 100$ points. It is stored as the tibble `wiggly` in the `Stat406` package. (Run `?wiggly` for details.)

```
data("wiggly")

ggplot(wiggly, aes(x, y)) +
  geom_point() +
  stat_function(fun = wiggly_function, linewidth = 0.5, linetype = "dashed") +
  labs(title = "Fixed Wiggly Data", x = "x", y = "y") +
  theme_bw()
```



We will re-explore our bias-variance analysis from Homework 2, but using a bootstrapped training sample from a fixed dataset rather than using truly independent training samples.

You'll need these helper functions from Homework 2:

```
#' Trains a polynomial regression model on training data and outputs
#' predictions on test data.
#' @param train.data A data frame containing a training sample with
#' columns `x` and `y`.
#' @param test.data A data frame containing the test data (e.g. `test.set`)
#' with columns `x`.
#' @param degree An integer specifying the degree of the polynomial to fit.
#' @return A vector containing the predictions for the test data.
compute.polynomial.predictions <- function(train.data, test.data, degree) {
  model <- lm(y ~ poly(x, degree), data = train.data)
  predictions <- predict(model, newdata = test.data)
  predictions
}

test.set <- tibble(x = seq(0.1, 6, length.out = 50))
test.set$y <- wiggly_function(test.set$x)
```

2.1 (1 point)

Recreate the dataframe `wiggly.preds` from Homework 2 Question 2.3, except instead of using `create.training.sample` to generate training samples, construct $B = 100$ bootstrap samples from the fixed `wiggly` data.

(Hint: see `?slice_sample` for help constructing bootstrap samples.)

Important: for full credit, the bootstrapped training sample:

- Must be of size $n = 100$ (preferably using `n=nrow(wiggly)` to avoid using a magic number)
- Must have been constructed using resampling (i.e. `replace = TRUE`)

```
set.seed(12345)  # Lock the seed for reproducibility

# your code
```

2.2 (1.5 points)

a. (0.5 points) Now that you have the `wiggly.preds` dataframe, compute the same bias/variance plot that we created in Homework 2 Question 2.4, but this time using the bootstrap training samples rather than true training samples.

b. (1 point) How do the trends in this plot differ from your Homework 2 plot? Offer potential hypotheses for why you may see different trends.

a.
b.

2.3 (1.5 points)

a. (1 point) Using the `wiggly.preds` dataframe you just created, use the nonparametric bootstrap to produce 95% confidence intervals for the degree 5 polynomial predictions at the first 10 test points. Store the results in a dataframe `wiggly.degree5.cis` with columns `x` (test input), `y` (true response), `lower` (lower CI value), and `upper` (upper CI value). **Print the output of the dataframe.**

Remember to correct for the fact that you are producing 10 confidence intervals.

b. (0.5 points) Compute the coverage of the confidence intervals you just produced. Does this number seem reasonable to you? Why or why not?

a.
b.

3 Bakeoff (3 points + potential bonus)

This exercise is based on the the TV show "The Great British Bakeoff". I am giving you information about bakers for the first 8 UK seasons. This data is derived from the package bakeoff by Alison Hill and Chester Ismay. You MAY NOT download this package (or use it). If you are unfamiliar with this show, I suggest you watch it! It's a great way to spend a few hours of your spare time.

A few other bits of important information. Again, easier if you just watch an episode or read Wikipedia, but nonetheless. Each episode consists of 3 "bakes": "signature challenge", "technical challenge", and "showstopper". In the technical, the bakers are ranked from best to worst. At the end of the episode, someone is awarded "star baker" (this week's winner), and someone goes home. Thus, the winners who make it to the last episode appear in more episodes. I have removed any information that depends on how long a baker lasts from the data. Also, in the first season, no one was awarded star baker. Because there is only one "overall winner" for the season, I have counted all the bakers that make the final episode as "winners". This means about a quarter of bakers are winners and the rest are losers.

> Your goal is to predict who will win and determine which features are useful for predicting the winners. "Winners" for the purposes of this Homework are defined to be any contestants involved in the final episode of the season (3 per season), rather than just the overall winner.

First, load the training data.

Notes: There are 4 character variables that cannot be used for this analysis because of their type. Do not use them. (They could likely be used with careful processing, but I doubt very much that it is worth the effort. For this reason, I'm leaving them in the data in case you want them for the bonus. But also because real data often comes with useless cruft you should probably remove.)

```
data("bakeoff_train")
```

3.1 (0.5 points)

In the data (after removing the 4 variables described in the note above), there are 2 `numeric` variables which are entirely useless. They measure something which has nothing to do with winners and losers, and they are perfectly correlated with other variables. What are they?

Your answer

3.2 (0.5 points)

There is one variable which is `numeric`, but should be treated as `categorical`. You must determine which, and treat it as `categorical` in all your models. Which is it?

Your answer

3.3 (0.5 points)

Estimate a random forest to predict "winners". You must use 400 trees. Report the Out-of-Bag error rate of the full 400 tree ensemble. You likely need to set the `na.action` argument to `randomForest()` to `na.omit`.

```
# your code
```

3.4 (0.5 points)

Inspect the Random Forest object. It contains a number of different things. Show the confusion matrix (part of the object). Does the Forest do better for winners or losers for the training set?

```
# your code
```

3.5 (0.5 point)

Describe the difference between Out-of-Bag error and In-Sample error.

Your answer

3.6 (0.5 points)

Using the same data, perform Bagging with 400 Trees. Report the confusion matrix. If your goal is "prediction accuracy" which method (RF or Bagging) should you use?

```
# your code
```

Bonus

There is also a `bakeoff_test` dataset. This data is missing the response column. Your goal is to predict the response as accurately as possible.

The reward.

The student with the best score will receive 2 extra points toward their final mark. The student with the second best score will receive 1 point. In the event of a first place tie (to 5 decimal places), all tied students will receive the first place bonus, but no second place will be awarded. In the event of a second place tie, all tied students will receive the second-place bonus.

Rules:

1. You must correctly upload one (and only 1) submission to the `bakeoff-bakeoff` repo. This must be done by a separate PR (from your usual HW submission). I would let you upload directly to `main`, but that would allow you to potentially delete submissions and sabotage your classmates.
2. Your submission must consist solely of a single `.rds` file. The name of the file must be `bake-ghusername.rds`. That file will contain 1 (one) `R` object which is a vector of `TRUE` and `FALSE` with your predicted classes. It must be of type `logical`.
3. Suppose you have a vector `preds = c(TRUE, FALSE)`. You can achieve the above with the code `saveRDS(preds, "bake-ghusername.rds")`.
4. Neither I nor the TAs will provide **any** assistance. This means no questions on Slack or in office hours about approaches or methods for the bonus will be answered. You may use any methods you like. We will not resolve your Github issues on this repo nor will any late submissions be allowed. If your data is in the wrong format, you are ineligible.
5. I have taken precautions to avoid cheating. If cheating is suspected, you will be disqualified with no recourse (i.e., don't come and argue with me).
6. Have fun.